

LAB 02. PART1: ARITHMETICS & LOGICAL OPERATIONS

Computer Vision

2025 - 2026

Overview

This lab is a hands-on introduction to two foundational families of digital image processing operations:

- **Part 1 (Pixel-wise operations):** arithmetic operations, logical operations, and Otsu automatic thresholding.
- **Part 2 (2D Transformation):** 2D transformation.

Throughout the lab, you will implement each method in two ways:

- **(A) NumPy-only:** directly process the image matrix and convert back to an image.
- **(B) Library-based (OpenCV):** use built-in functions for comparison and validation.

Learning Objectives

After completing this lab, students will be able to:

- Treat a grayscale image as a 2D intensity matrix and implement pixel-wise transformations.
- Explain and apply gamma correction, arithmetic image blending, and logical masking.
- Implement Otsu thresholding from scratch and interpret when it works (and fails).
- Explain the meaning of 2D convolution and implement it for both VALID and SAME modes.
- Build simple image-processing pipelines combining multiple operations toward a goal.

Environment Setup

```
pip install numpy opencv-python matplotlib
```

Important Notes (Read Before Coding)

- Most images are loaded as `uint8` in `[0, 255]`. If you add/subtract with NumPy directly on `uint8` without care, **overflow/underflow may occur**.
- A safe practice for custom math is: convert to `float32` in `[0, 1]` → compute → clip → convert back to `uint8`.
- Logical operations (AND/OR/XOR) are most meaningful on **binary images** (0 or 255). In this lab, we will binarize first (manual threshold or Otsu).
- Convolution outputs may contain negative values or exceed 255 (e.g., edge filters). For visualization, you may need **normalization** or **clipping**.

Common Utilities (Shared by All Tasks)

Purpose: These helper functions standardize reading, type conversion, visualization, and saving.

```
import numpy as np
import cv2
import matplotlib.pyplot as plt

def read_gray_cv2(path: str) -> np.ndarray:
    """Read a grayscale image -> uint8 array [H,W] in [0..255]."""
    img = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
    if img is None:
        raise FileNotFoundError(f"Cannot read image: {path}")
    return img

def to_float01(img_u8: np.ndarray) -> np.ndarray:
    """uint8 [0..255] -> float32 [0..1]."""
    return img_u8.astype(np.float32) / 255.0

def to_uint8(img_float01: np.ndarray) -> np.ndarray:
    """float [0..1] -> uint8 [0..255] (clip)."""
    x = np.clip(img_float01, 0.0, 1.0)
    return (x * 255.0 + 0.5).astype(np.uint8)

def show_gray(img_u8: np.ndarray, title: str = "") -> None:
    """Display grayscale image via matplotlib."""
    plt.figure()
    plt.imshow(img_u8, cmap='gray', vmin=0, vmax=255)
    plt.title(title)
    plt.axis('off')
    plt.show()

def save_gray(path: str, img_u8: np.ndarray) -> None:
    """Save grayscale image."""
    cv2.imwrite(path, img_u8)
```

PART 1. ARITHMETIC & LOGICAL OPERATIONS + OTSU

0.1 Introduction

In Part 1, we treat a grayscale image as a 2D matrix of intensities. All operations here are **pixel-wise** (a.k.a. point operations):

$$\text{output}(x, y) = f(\text{input}(x, y))$$

That is, each pixel is processed **independently**, without using neighbors.

0.2 Arithmetic Operations on Images

0.2.1 Why Arithmetic Operations? (Conceptual Motivation)

Arithmetic operations can be used to:

- adjust global brightness/contrast (e.g., blending, scaling),
- emphasize differences between two images (subtraction, absolute difference),
- combine information from multiple sources (addition, weighted sum),
- normalize by a reference image or illumination field (division).

0.2.2 Standard Practical Workflow (Step-by-step)

For most arithmetic tasks in this lab, follow this workflow:

1. Read image(s) $\rightarrow$ grayscale.
2. Treat image as a 2D array of intensities.
3. Convert to float in $[0, 1]$ for safe math (NumPy version).
4. Apply the arithmetic transform (possibly with parameters).
5. Clip/normalize results into a valid displayable range.
6. Convert back to `uint8` and display/save.

0.2.3 Gamma Correction (Power-law Transform)

Theory (Short): Gamma correction is a non-linear intensity transform:

$$s = r^\gamma, \quad r \in [0, 1]$$

Interpretation:

- $\gamma < 1$ expands dark tones (brightens shadows) and compresses highlights.
- $\gamma > 1$ compresses shadows and expands highlights (overall darker).
- $\gamma = 1$ leaves the image unchanged.

When to use:

- Improve visibility in underexposed images ($\gamma < 1$).
- Reduce blown-out highlights ($\gamma > 1$).
- As preprocessing before thresholding/segmentation to separate modes in histogram.

Caution: If the image has strong noise in dark regions, using $\gamma < 1$ can amplify that noise.

NumPy Implementation

```
def gamma_correction_np(img_u8: np.ndarray, gamma: float) -> np.ndarray:
    """
    Gamma correction:  $s = r^\gamma$  ( $r$  in  $[0,1]$ ).
    gamma < 1: brighten shadows; gamma > 1: darken highlights.
    """
    if gamma <= 0:
        raise ValueError("gamma must be > 0")
    r = to_float01(img_u8)
    s = np.power(r, gamma)
    return to_uint8(s)
```

OpenCV Implementation (LUT-based) **Note:** We keep the same mapping as NumPy: $s = r^\gamma$.

```
def gamma_correction_cv2(img_u8: np.ndarray, gamma: float) -> np.ndarray:
    if gamma <= 0:
        raise ValueError("gamma must be > 0")
    table = np.array([(i/255.0) ** gamma) * 255.0 for i in range(256)], dtype=np.uint8)
    return cv2.LUT(img_u8, table)
```

Mini-Task (Do it)

1. Load one image `input.png`.
2. Try $\gamma \in \{0.5, 0.8, 1.2, 2.0\}$.
3. Compare results and answer: which regions change the most?

0.2.4 Addition / Subtraction / Multiplication / Division

Theory (Short): These are element-wise operations. For two images A and B :

$$S_{\text{add}} = \alpha A + \beta B, \quad S_{\text{sub}} = \alpha A - \beta B, \quad S_{\text{mul}} = A \odot B, \quad S_{\text{div}} = A \oslash (B + \epsilon)$$

Practical meanings:

- **Addition:** blending / increasing brightness.
- **Subtraction:** change detection; highlight differences.

- **Multiplication:** emphasize common bright/dark regions (acts like masking in continuous intensity).
- **Division:** normalize by a reference (e.g., illumination correction); can strongly amplify values.

NumPy Implementations (Safe Float Workflow)

```
def add_images_np(a_u8: np.ndarray, b_u8: np.ndarray, alpha: float = 1.0, beta: float = 1.0) -> np.ndarray:
    """S = alpha*A + beta*B on [0,1], then clip."""
    A, B = to_float01(a_u8), to_float01(b_u8)
    S = alpha * A + beta * B
    return to_uint8(S)

def sub_images_np(a_u8: np.ndarray, b_u8: np.ndarray, alpha: float = 1.0, beta: float = 1.0) -> np.ndarray:
    """S = alpha*A - beta*B on [0,1], then clip."""
    A, B = to_float01(a_u8), to_float01(b_u8)
    S = alpha * A - beta * B
    return to_uint8(S)

def mul_images_np(a_u8: np.ndarray, b_u8: np.ndarray, scale: float = 1.0) -> np.ndarray:
    """S = scale*(A*B) on [0,1], then clip."""
    A, B = to_float01(a_u8), to_float01(b_u8)
    S = scale * (A * B)
    return to_uint8(S)

def div_images_np(a_u8: np.ndarray, b_u8: np.ndarray, eps: float = 1e-6, scale: float = 1.0) -> np.ndarray:
    """
    S = scale*(A/(B+eps)) on [0,1], then clip.
    Division may exceed 1 -> clipping may lose info; consider
    normalization if needed.
    """
    A, B = to_float01(a_u8), to_float01(b_u8)
    S = scale * (A / (B + eps))
    return to_uint8(S)
```

OpenCV Implementations (Saturating Arithmetic)

```
def add_images_cv2(a_u8: np.ndarray, b_u8: np.ndarray) -> np.ndarray:
    return cv2.add(a_u8, b_u8) # saturation to [0..255]

def sub_images_cv2(a_u8: np.ndarray, b_u8: np.ndarray) -> np.ndarray:
    return cv2.subtract(a_u8, b_u8)

def mul_images_cv2(a_u8: np.ndarray, b_u8: np.ndarray, scale: float = 1.0) -> np.ndarray:
    return cv2.multiply(a_u8, b_u8, scale=scale)

def div_images_cv2(a_u8: np.ndarray, b_u8: np.ndarray, scale: float = 1.0) -> np.ndarray:
```

```
b_safe = np.maximum(b_u8, 1).astype(np.uint8)
return cv2.divide(a_u8, b_safe, scale=scale)
```

Mini-Task (Do it)

1. Load two same-size grayscale images A and B .
2. Compute $A+B$, $A-B$, $A*B$, $A/(B+\text{eps})$ using both NumPy and OpenCV.
3. Discuss differences between NumPy (float+clip) and OpenCV (saturating integer ops).

Guiding Questions

- Why is subtraction useful for “change detection”?
- Why can division easily produce large values?
- In what cases does clipping hide meaningful information?

0.3 Logical Operations on Images (AND / OR / XOR)

0.3.1 Motivation

Logical operations are most meaningful for **binary masks**:

- combine multiple masks (OR),
- keep only the intersection of conditions (AND),
- detect disagreements/differences between masks (XOR),
- extract Region of Interest (ROI) by masking the original image.

0.3.2 Practical Workflow

1. Produce binary images (manual threshold or Otsu).
2. Apply AND/OR/XOR on binary results.
3. Optionally apply the binary mask to the grayscale image to extract ROI.

0.3.3 Code (Manual Binary Threshold)

```
def threshold_binary_np(img_u8: np.ndarray, t: int) -> np.ndarray:
    """Binary image: pixel >= t -> 255 else 0."""
    return np.where(img_u8 >= t, 255, 0).astype(np.uint8)
```

0.3.4 Code (NumPy Logical Ops)

```
def and_images_np(a_bin: np.ndarray, b_bin: np.ndarray) -> np.ndarray:
    return np.where((a_bin == 255) & (b_bin == 255), 255, 0).astype(np.
        uint8)

def or_images_np(a_bin: np.ndarray, b_bin: np.ndarray) -> np.ndarray:
    return np.where((a_bin == 255) | (b_bin == 255), 255, 0).astype(np.
        uint8)

def xor_images_np(a_bin: np.ndarray, b_bin: np.ndarray) -> np.ndarray:
    return np.where((a_bin == 255) ^ (b_bin == 255), 255, 0).astype(np.
        uint8)
```

0.3.5 Code (OpenCV Bitwise Ops)

```
def and_images_cv2(a_bin: np.ndarray, b_bin: np.ndarray) -> np.ndarray:
    return cv2.bitwise_and(a_bin, b_bin)

def or_images_cv2(a_bin: np.ndarray, b_bin: np.ndarray) -> np.ndarray:
    return cv2.bitwise_or(a_bin, b_bin)

def xor_images_cv2(a_bin: np.ndarray, b_bin: np.ndarray) -> np.ndarray:
    return cv2.bitwise_xor(a_bin, b_bin)
```

Mini-Task (Do it)

1. Create two masks from the same image using two different thresholds.
2. Compute AND/OR/XOR between them.
3. Explain which operator is suitable for: “keep only confident pixels” vs “merge all candidates”.

0.4 Otsu Automatic Thresholding

0.4.1 Motivation

Manual thresholding requires choosing a threshold by trial-and-error. Otsu automatically selects a global threshold based on the image histogram, aiming to separate the image into two classes (background vs foreground).

0.4.2 Key Idea (Short Theory)

Otsu chooses threshold t^* that maximizes **between-class variance**:

$$\sigma_b^2(t) = \omega_0(t) \omega_1(t) (\mu_0(t) - \mu_1(t))^2$$

where:

- ω_0, ω_1 are probabilities (class weights),
- μ_0, μ_1 are class means.

This works best when the histogram is close to bimodal (two peaks).

0.4.3 When Otsu Works / Fails

Works well when:

- near-bimodal histogram,
- relatively uniform illumination,
- moderate noise.

May fail when:

- non-uniform lighting (shadows/gradients),
- strong overlap between foreground and background intensities,
- object occupies a very tiny portion of the image,
- histogram not bimodal.

0.4.4 NumPy Implementation

```
def otsu_threshold_np(img_u8: np.ndarray) -> int:
    """
    Returns Otsu threshold t* in [0..255].
    Implementation based on between-class variance.
    """
    hist = np.bincount(img_u8.ravel(), minlength=256).astype(np.float64)
    N = hist.sum()
    if N == 0:
        return 0

    p = hist / N
    omega = np.cumsum(p)
    mu = np.cumsum(p * np.arange(256))
    mu_T = mu[-1]

    eps = 1e-12
    w0 = omega
    w1 = 1.0 - omega
    mu0 = mu / (w0 + eps)
    mu1 = (mu_T - mu) / (w1 + eps)

    sigma_b2 = w0 * w1 * (mu0 - mu1) ** 2
    t_star = int(np.argmax(sigma_b2))
    return t_star

def otsu_binarize_np(img_u8: np.ndarray):
    t = otsu_threshold_np(img_u8)
    bin_img = threshold_binary_np(img_u8, t)
    return t, bin_img
```

0.4.5 OpenCV Implementation

```
def otsu_binarize_cv2(img_u8: np.ndarray):
    t, bin_img = cv2.threshold(img_u8, 0, 255, cv2.THRESH_BINARY + cv2.
        THRESH_OTSU)
    return int(t), bin_img
```

Mini-Task (Do it)

1. Apply Otsu on the original image.
2. Apply gamma correction first, then Otsu.
3. Compare thresholds and results. Explain why gamma can make Otsu easier (or harder).

0.5 Integrated Exercises After Part 1 (Pipelines)

These exercises combine multiple operations toward a practical goal.

Exercise 1: Brighten then Otsu

Goal: Improve segmentation on a dark image.

1. Gamma with $\gamma < 1$ (e.g., 0.6).
2. Otsu thresholding on gamma-corrected image.
3. Compare with Otsu on the original image.

Exercise 2: ROI Extraction by AND

Goal: Extract foreground region only.

1. Otsu to get mask.
2. Bitwise AND between mask and original image to keep ROI.

Exercise 3: Change Detection (Subtraction + Otsu)

Goal: Detect changed regions between two aligned images.

1. Compute absolute difference: $|A - B|$.
2. Apply Otsu to difference image to get change mask.

Exercise 4: Multi-step Mask Fusion (Logic)

Goal: Fuse two masks obtained from different thresholds or preprocessing.

1. Create `mask1` (e.g., Otsu on original).
2. Create `mask2` (e.g., Otsu after gamma).
3. Combine: OR to merge candidates, AND to keep consensus, XOR to find disagreements.